

Namwook Lee — CS 180, UC Berkeley, Fall 2026 — Project 2: Fun with Filters and Frequencies
URL: <https://namwook0921.github.io/proj2/>



namwook_lee

Project 2 · Fun with Filters and Frequencies

cal-cs180.github.io/fa26/hw/proj2

9 posts **180** friends **2** cameras

1.1 Convolutions from Scratch

1.2 Finite Difference Operator

1.3 Derivative of Gaussian Filter

2.1 Image Sharpening

2.2 Derek & Nutmeg

2.2 King & LeBron

2.2 Hybrid Images

2.3 Gaussian and Laplacian Stacks

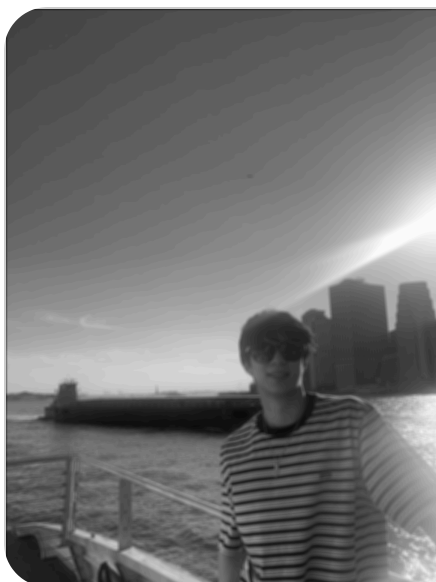
2.4 Multiresolution Blending

TAP THE SMALL FRAME IN ANY POST TO SWAP THE TWO VIEWS · CLICK A PHOTO TO ENLARGE · ← → TO MOVE BETWEEN PHOTOS

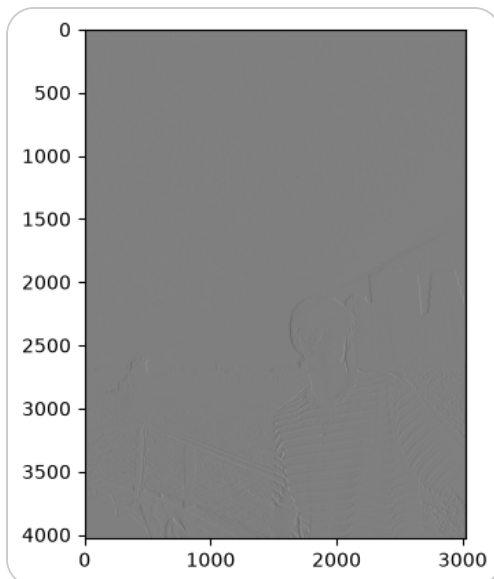
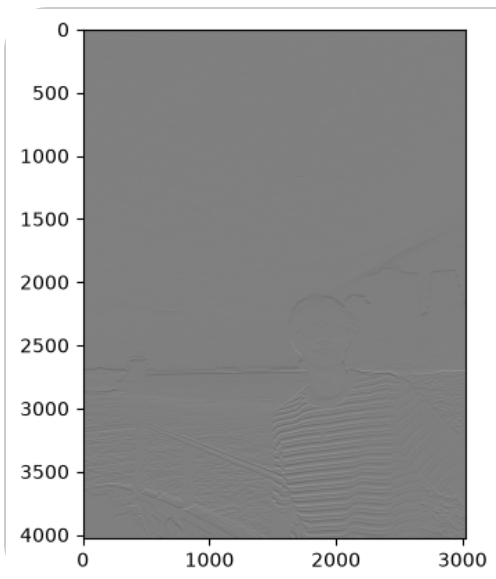
**namwook_lee**

Four loops, then two

PART 1.1



9x9 box filter



Dy

Since I wore a shirt with horizontal stripes, Dy shows more details.

namwook_lee Convolutions from Scratch

FOUR LOOPS

```
def convolution_4d(img, kernel, padding=None):
    h, w = kernel.shape
    m, n = img.shape
    kernel = kernel[::-1,::-1]

    if padding == "same":
        pad_h = h // 2
        pad_w = w // 2
    elif padding == "full":
        pad_h = h - 1
```

```

    pad_w = w - 1

    img_pad = np.pad(img, ((pad_h, pad_h), (pad_w, pad_w)))

    h_out = img_pad.shape[0] - h + 1
    w_out = img_pad.shape[1] - w + 1

    img_out = np.zeros(shape=(h_out, w_out))

    for i in range(h_out):
        for j in range(w_out):
            val = 0
            for k in range(h):
                for l in range(w):
                    val += kernel[k][l] * img_pad[i+k][j+l]
            img_out[i][j] = val

    return img_out

```

The four-loop convolution does a 2D loop across the offset of the kernel to the image's origin, and does another 2D loop inside it across each element of the kernel. For a random 3×3 kernel, it takes 73.2 seconds.

TWO LOOPS

```

def convolution_2d(img, kernel, padding=None):
    h, w = kernel.shape
    m, n = img.shape
    kernel = kernel[::-1,::-1]

    if padding == "same":
        pad_h = h // 2
        pad_w = w // 2
    elif padding == "full":
        pad_h = h - 1
        pad_w = w - 1

    img_pad = np.pad(img, ((pad_h, pad_h), (pad_w, pad_w)))

    h_out = img_pad.shape[0] - h + 1
    w_out = img_pad.shape[1] - w + 1

    img_out = np.zeros(shape=(h_out, w_out))

    for i in range(h_out):
        for j in range(w_out):
            img_out[i][j] = np.sum(kernel * img_pad[i:i+h,j:j+w])

    return img_out

```

The two-loop convolution reduces this by replacing the latter loop with an element-wise multiplication. For a random 3×3 kernel, it takes 47.3 seconds.

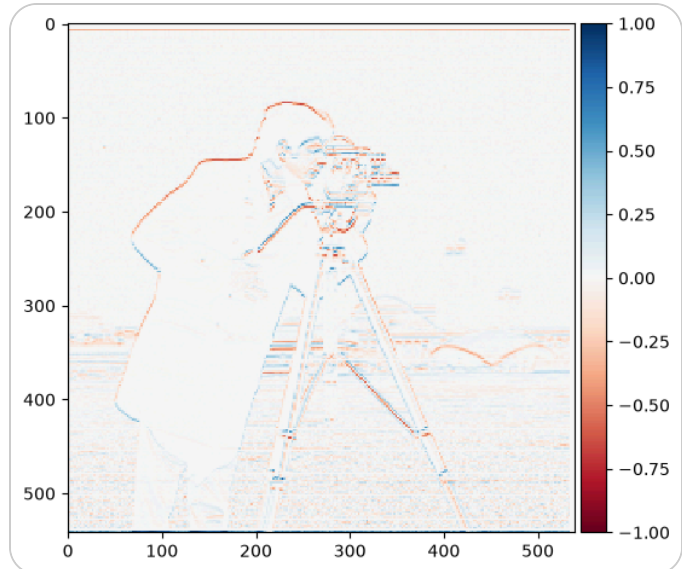
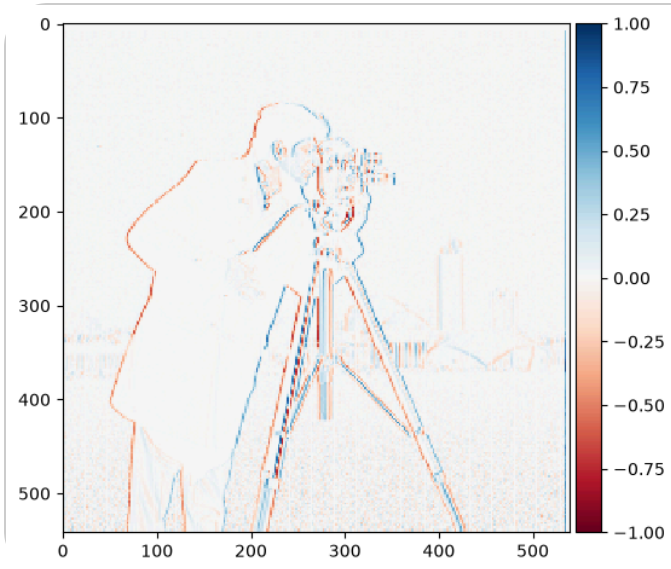
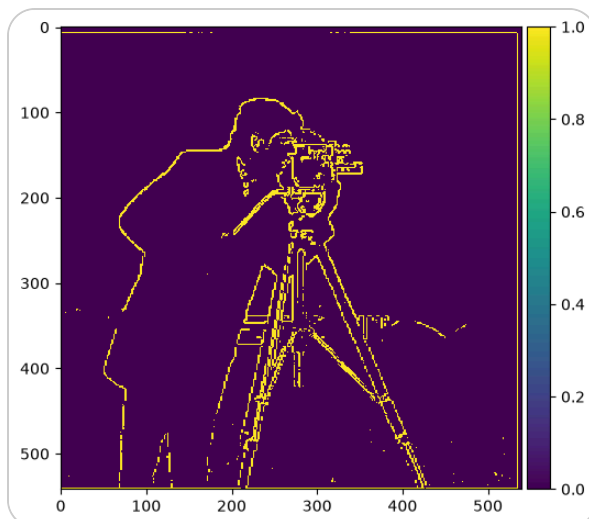
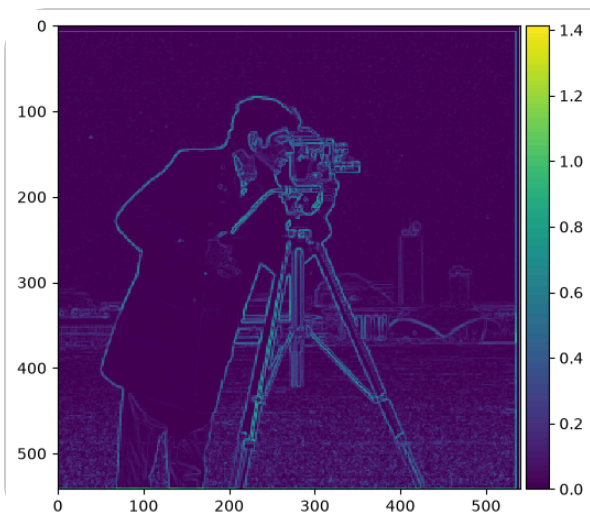
TIMING

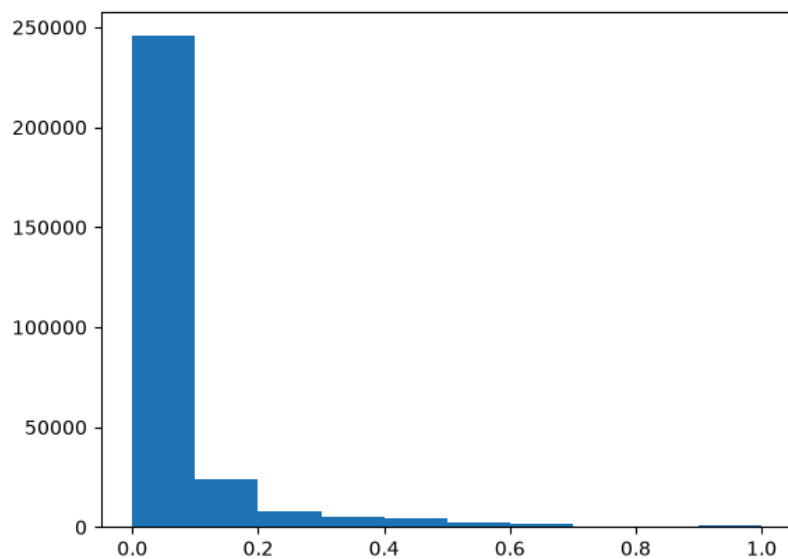
Method	Runtime	Speed-up over four loops
Four loops	73.2 s	1×
Two loops	47.3 s	1.5×
<code>scipy.signal.convolve2d</code>	2.9 s	25×

PART 1.1 · SEPTEMBER 2026



namwook_lee
cameraman.png

PART 1.2**Dx****Gradient
magnitude**



namwook_lee Finite Difference Operator

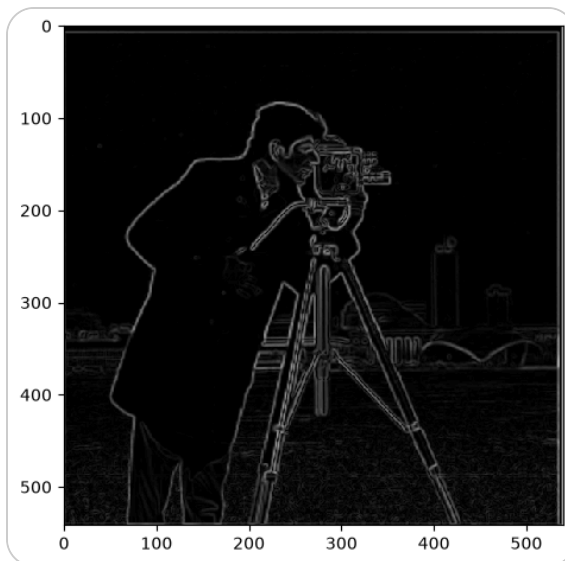
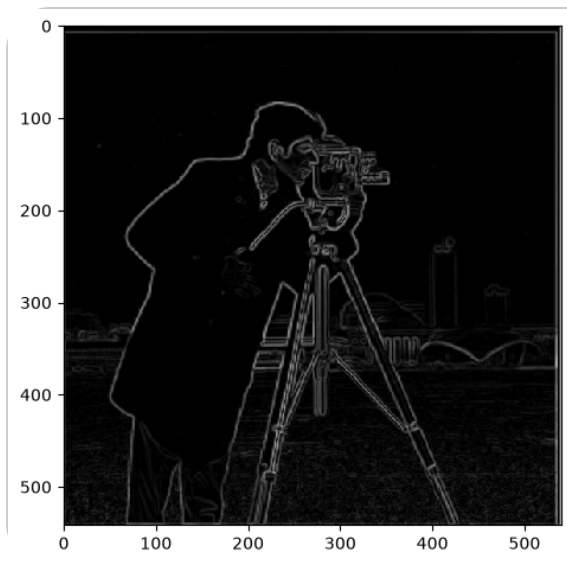
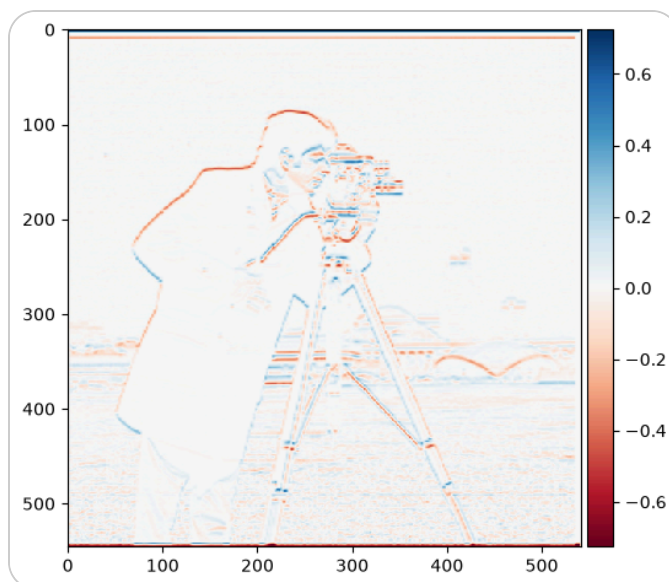
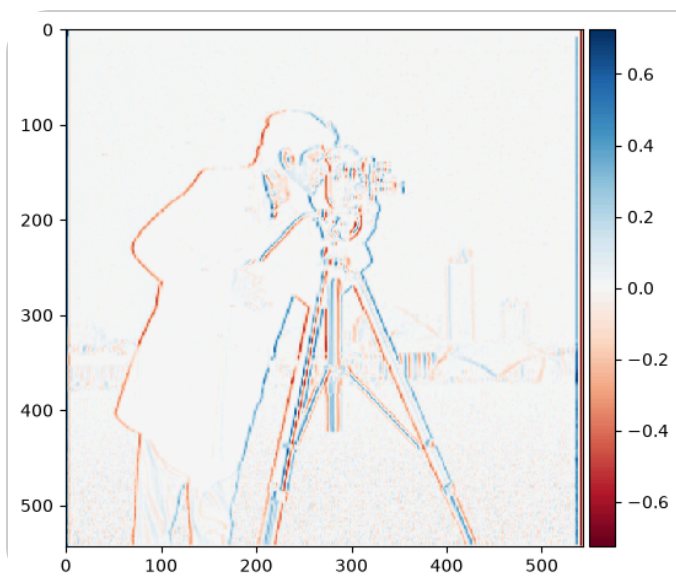
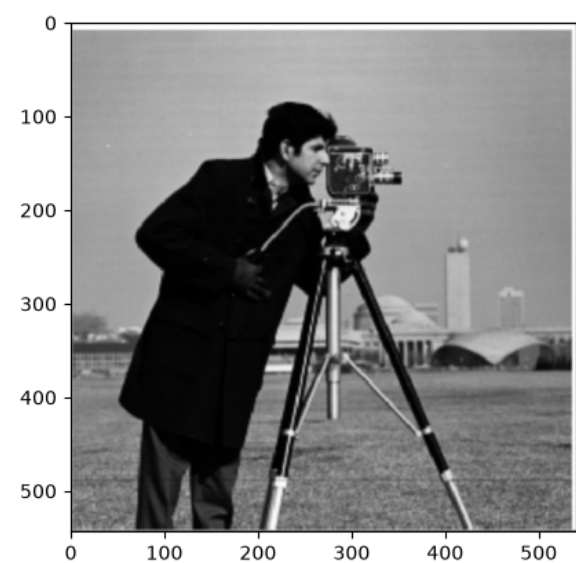
The histogram shows most of the value of the pixels lie within 0.1, and about a tenth of it in 0.2. Therefore, I tried threshold = 0.2, but still included a lot of dots. Then, threshold 0.3 preserved most of the edges while getting rid of the dots.

PART 1.2 · SEPTEMBER 2026

**namwook_lee**

Blur first, then differentiate

PART 1.3

Blur, then
differentiate

Dx

namwook_lee Derivative of Gaussian Filter

Gaussian Blur $\rightarrow (D_x^2 + D_y^2)$ is exactly equal to $(DoG_x^2 + DoG_y^2)$ because convolution is associative. Numerically, max pixel difference between the two images constructed by blur $\rightarrow$ convolution vs DoG convolution is $3.74e-16$.

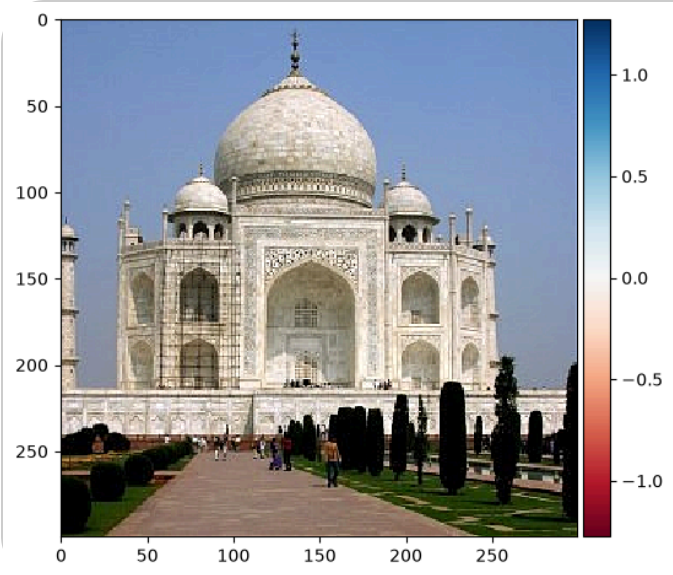
PART 1.3 · SEPTEMBER 2026



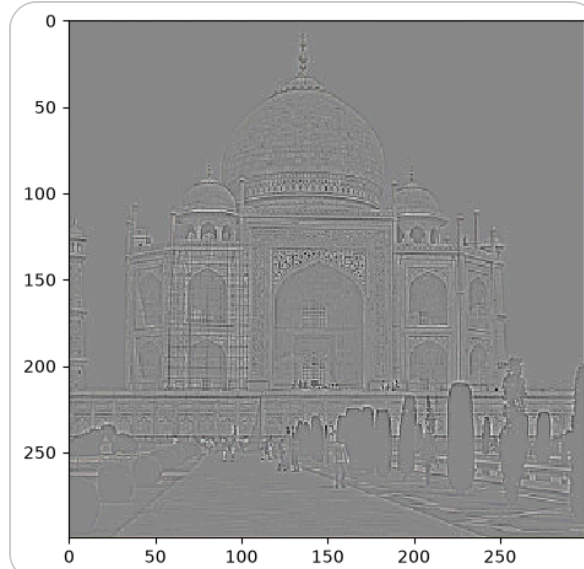
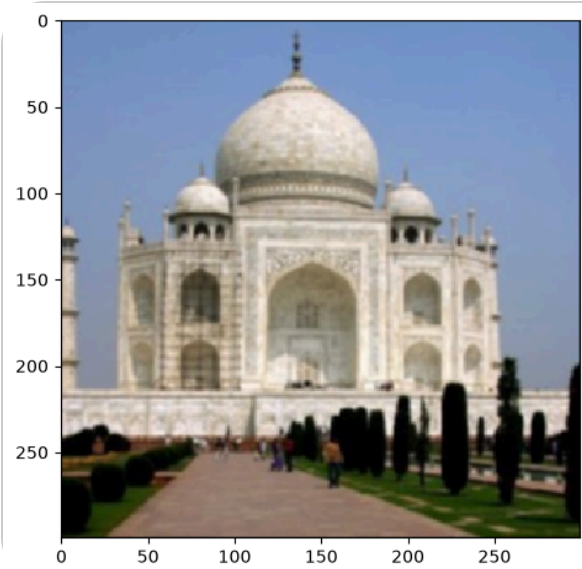
namwook_lee

Unsharp mask · Taj Mahal

PART 2.1



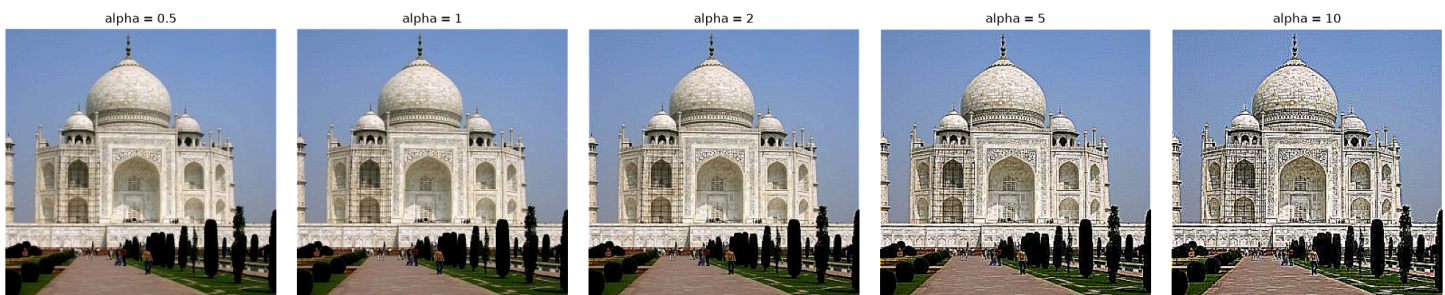
Sharpened



Gaussian blur

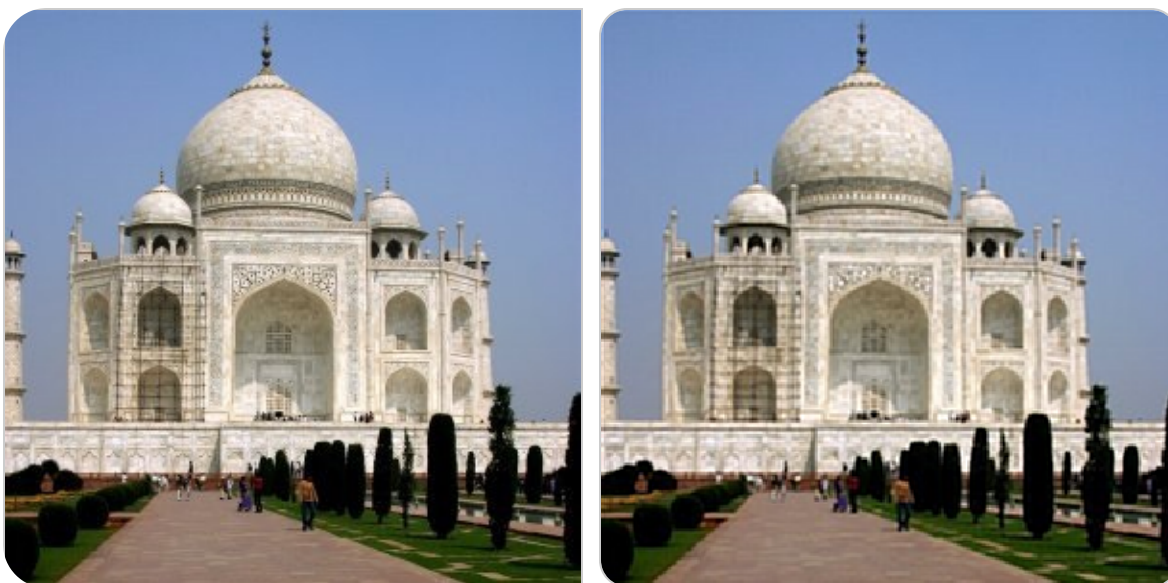
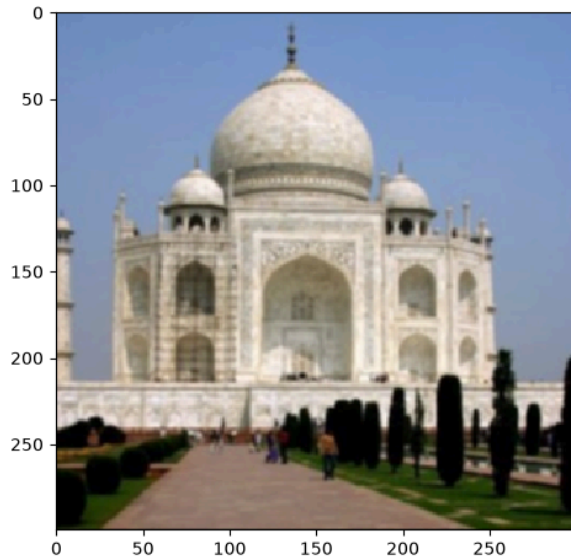
The unsharp masking is done by applying $I + \alpha * (I - G)$, where I is the identity operation and G is the Gaussian blur. Applying G to the image results in lowering the frequency, and deleting most of the high frequency details, so that the image is blurred.

Subtracting the Gaussian from the original image, therefore leaves the high frequency details, while reducing the low frequency aspects of the image.



(Click to zoom)

As $(I - G)$ applied to the image leaves the high frequency details of the image, adding this to the original image results in a sharpened version of the image. This is how the unsharpen mask works, and above is a result of sweeping alpha, a constant multiplied to $(I - G)$ when sharpening the image. Higher alpha leads to a more exaggerated sharpening of the image.



Original

The blurring destroys the high frequency details, so applying unsharp masking cannot salvage the loss of it.

namwook_lee Image Sharpening

PART 2.1 · SEPTEMBER 2026



namwook_lee
Derek & Nutmeg

PART 2.2



Derek

Derek & Nutmeg



Hybrid

The human eye focuses on the high frequency details when the image is closer, and to the lower frequencies when it's further. Therefore, you can find Nutmeg from the original size hybrid image (the screen is close to your eye!). For the zoomed out image, you barely notice Nutmeg, and the shape of Derek is more outstanding.

namwook_lee Derek & Nutmeg

PART 2.2 · SEPTEMBER 2026

**namwook_lee**

King & LeBron

PART 2.2



king



Hybrid

Real Recognize Real.

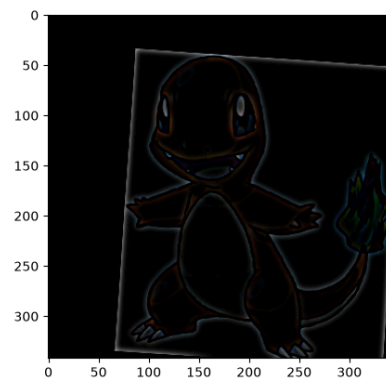
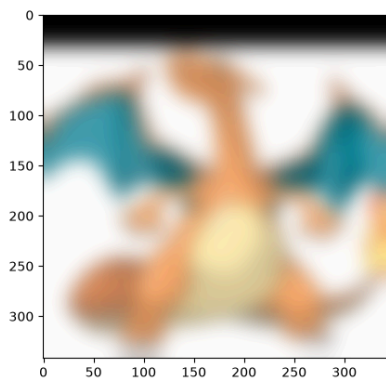
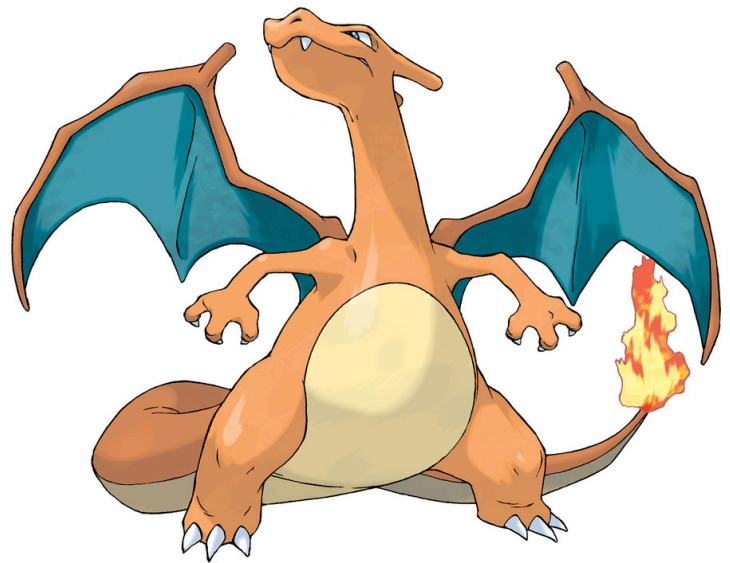
namwook_lee King & LeBron

PART 2.2 · SEPTEMBER 2026



namwook_lee
Charmander & Charizard

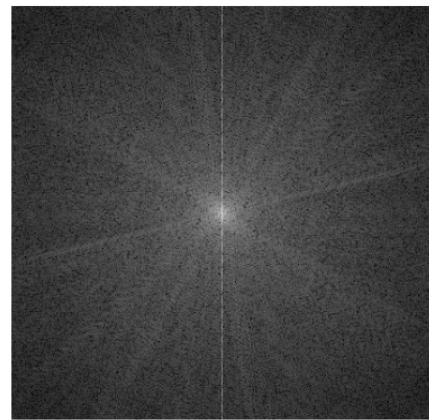
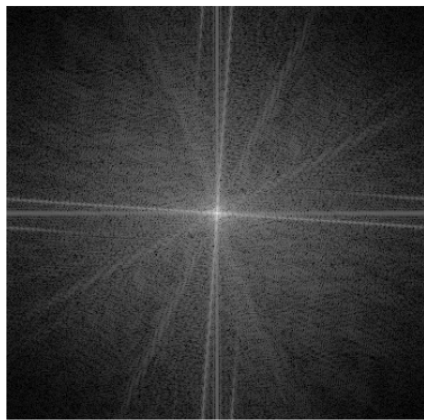
⚡ BOTH CAMERAS



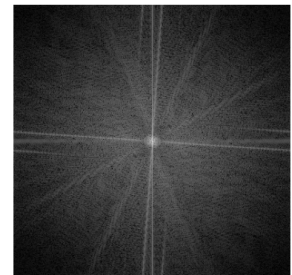
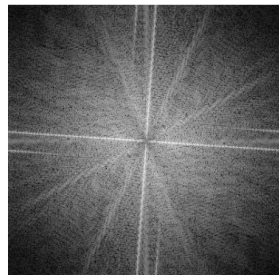
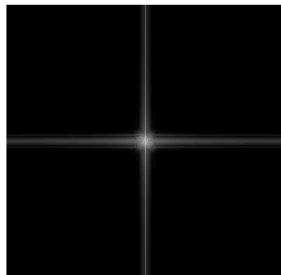
Low-pass ·
Charizard



Hybrid



FFT of each images.



Left is FFT of low pass charizard. Most of the mass is centered at the origin, because the images is blurred. Middle is FFT of high pass ($I - G$) of charmander. The mass is spread out, and center is hollow. Right is the FFT of the hybrid image, which resembles the distribution of the above 2 FFT of the original images, as it is averaged out.

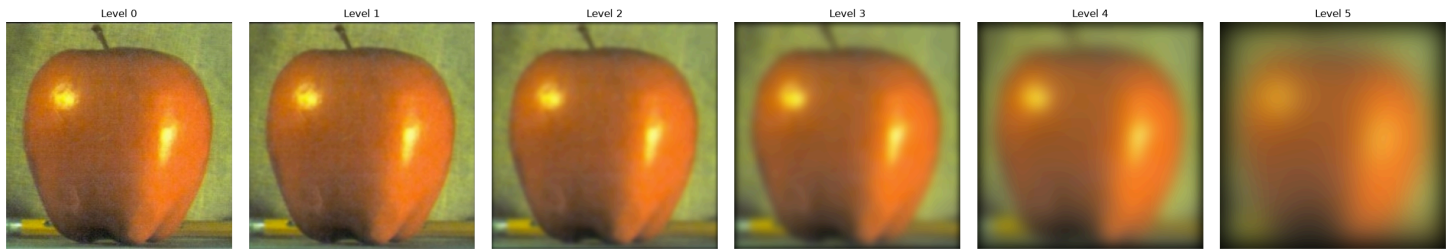
namwook_lee Hybrid Images

PART 2.2 · SEPTEMBER 2026

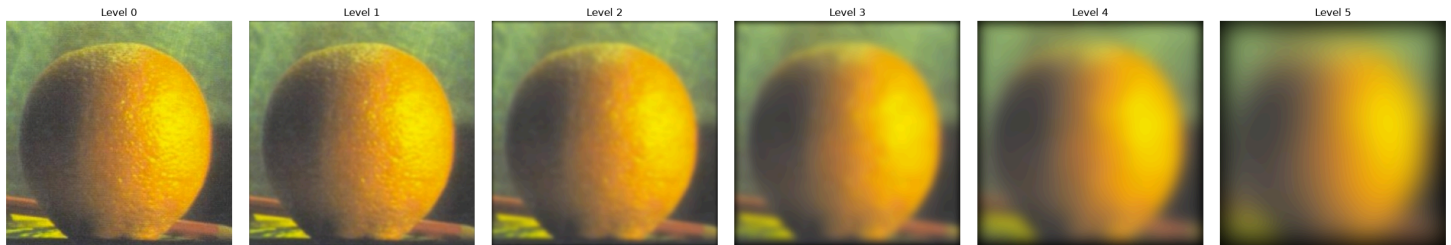
**namwook_lee**

Six levels, no downsampling

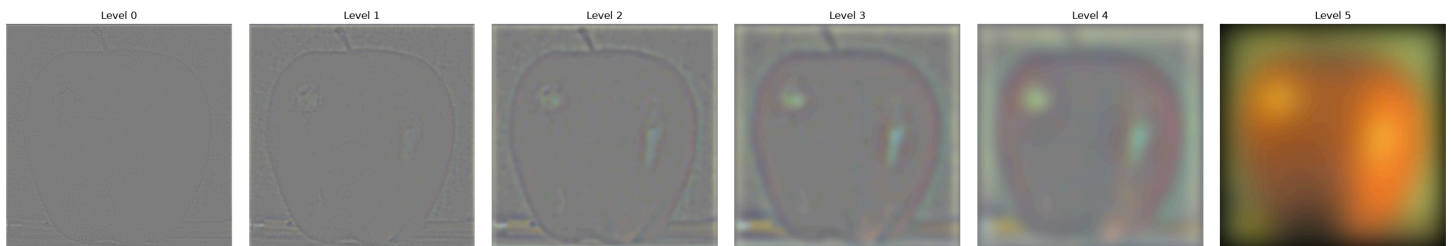
PART 2.3



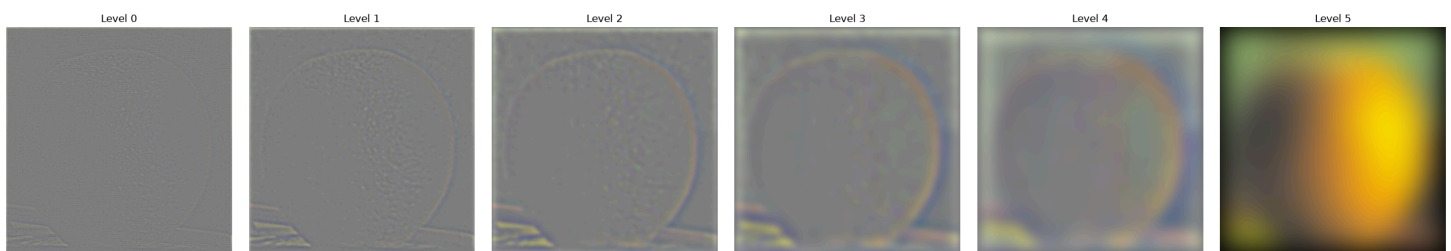
Gaussian stack — apple.



Gaussian stack — orange.



Laplacian stack — apple.



Laplacian stack — orange.

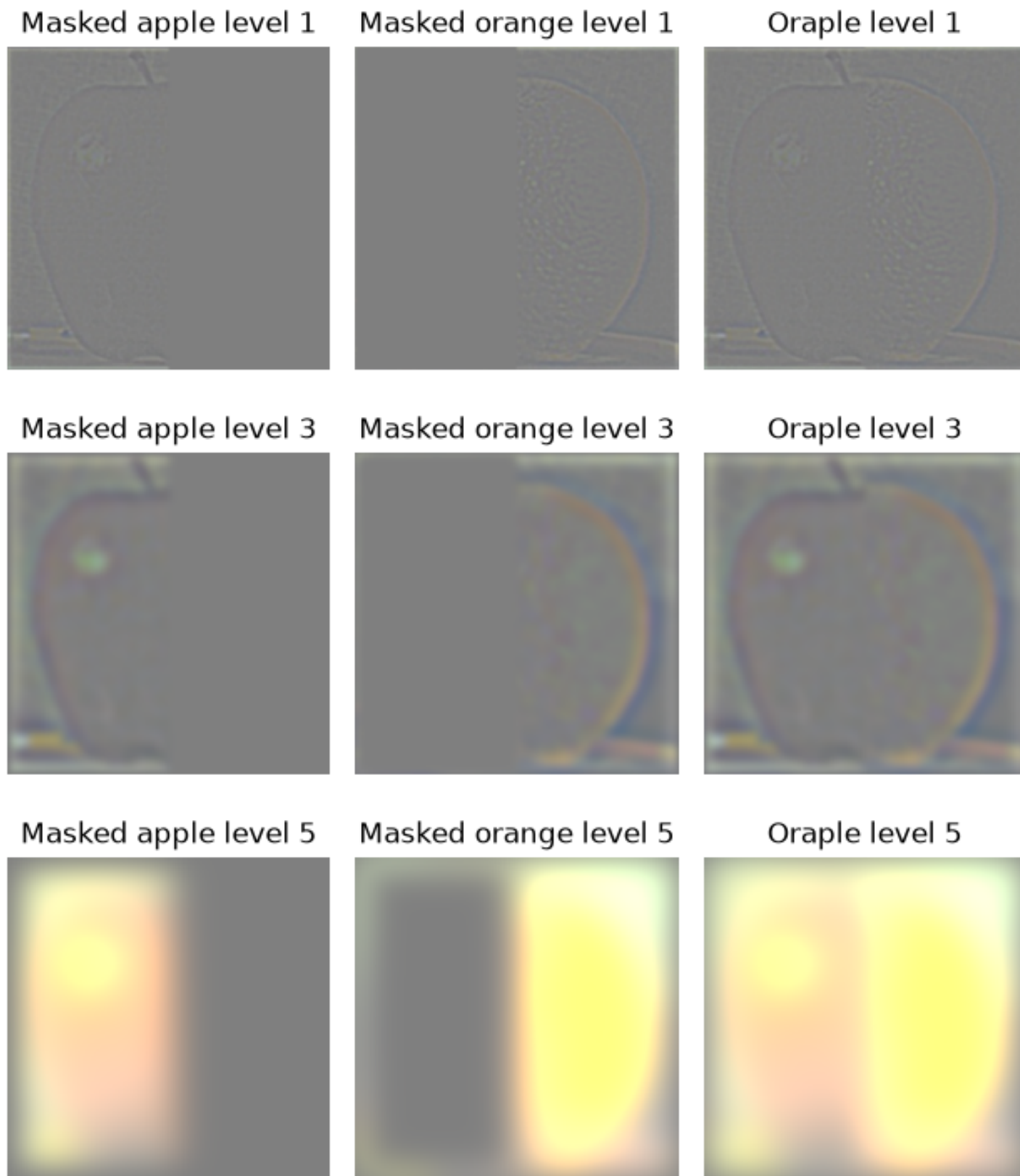


Figure 3.42, recreated at levels 1, 3 and 5.

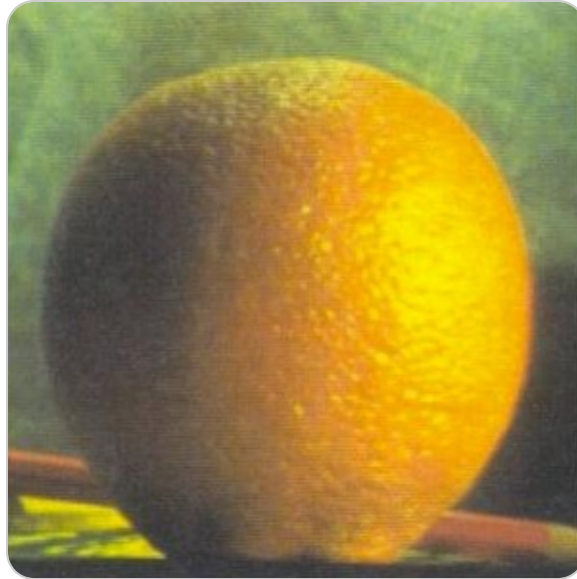
namwook_lee Gaussian and Laplacian Stacks

PART 2.3 · SEPTEMBER 2026

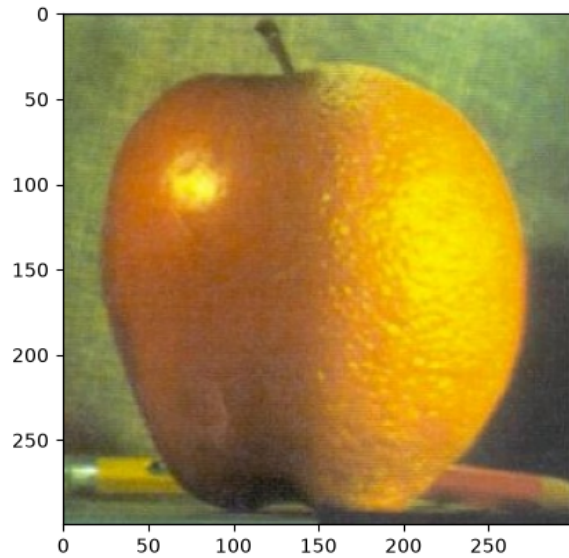
**namwook_lee**

The oraple, and friends

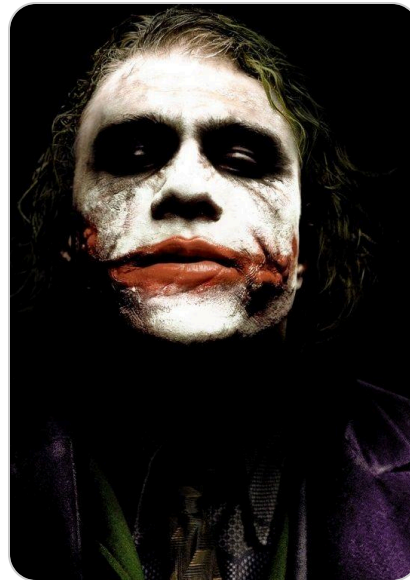
⚡ 4H LATE



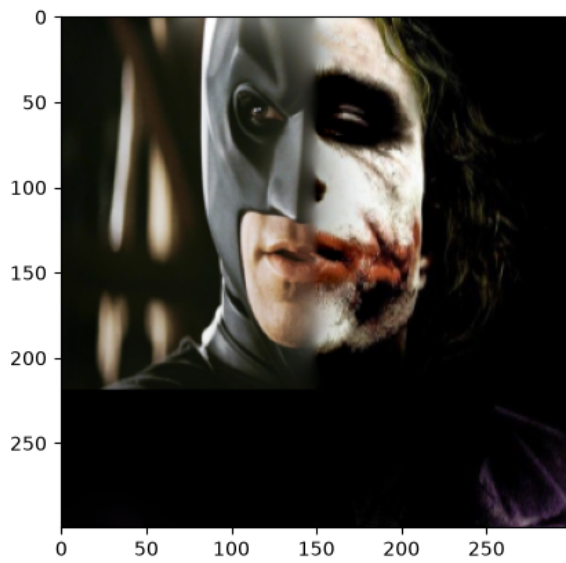
apple



I want to try this fruit.



batman



Goated movie, Goated archenemy



baby_me



I always wanted to star in Teletubbies!!

namwook_lee Multiresolution Blending

PART 2.4 · SEPTEMBER 2026

[Profile](#) · [Project 0](#) · [Project 1](#) · [Project 2](#) · [Project 2 spec](#) · [GitHub](#)

Namwook Lee © 2026 · CS 180, UC Berkeley